

OOP LAB – Properly Indented Solutions

1. Room Class

```
class Room {
    int no;
    String type;
    double area;
    boolean ac;

    void setData(int n, String t, double a, boolean acOn) {
        no = n;
        type = t;
        area = a;
        ac = acOn;
    }

    void display() {
        System.out.println(no + " " + type + " " + area + " AC:" + ac);
    }

    public static void main(String[] a) {
        Room r = new Room();
        r.setData(1, "Deluxe", 200, true);
        r.display();
    }
}
```

2.i Static Block

```
class S {
    static int x;

    static {
        x = 10;
        System.out.println("Static block");
    }

    static void show() {
        System.out.println(x);
    }

    public static void main(String[] a) {
        show();
    }
}
```

2.ii Greatest of 3 Numbers

```
import java.util.*;

class Max3 {
    public static void main(String[] a) {
        Scanner s = new Scanner(System.in);
        System.out.println(Math.max(s.nextInt(),
                                    Math.max(s.nextInt(), s.nextInt())));
    }
}
```

3.i Method Overriding

```
class A {
    void show() {
        System.out.println("A");
    }
}

class B extends A {
    void show() {
        System.out.println("B");
    }
}

class T {
    public static void main(String[] a) {
        A x = new B();
        x.show();
    }
}
```

3.ii Super Keyword

```
class P {
    int x = 10;

    void show() {
        System.out.println("P");
    }
}

class C extends P {
    int x = 20;

    void show() {
        System.out.println(x);
        System.out.println(super.x);
        super.show();
    }

    public static void main(String[] a) {
        new C().show();
    }
}
```

4. Polymorphism (Abstract Class)

```
abstract class Shape {
    abstract void draw();
    abstract void erase();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Draw Circle");
    }

    void erase() {
        System.out.println("Erase Circle");
    }
}

class T {
    public static void main(String[] a) {
        Shape s = new Circle();
        s.draw();
        s.erase();
    }
}
```

5. Inner Class

```
class Outer {
    void show() {
        System.out.println("Outer");
    }

    class Inner {
        void show() {
            System.out.println("Inner");
        }
    }

    public static void main(String[] a) {
        Outer o = new Outer();
        o.show();
        o.new Inner().show();
    }
}
```

6. Box + Box3D (Inheritance)

```

class Box {
    double l, b;

    Box(double l, double b) {
        this.l = l;
        this.b = b;
    }

    double area() {
        return l * b;
    }
}

class Box3D extends Box {
    double h;

    Box3D(double l, double b, double h) {
        super(l, b);
        this.h = h;
    }

    double volume() {
        return l * b * h;
    }

    public static void main(String[] a) {
        Box3D d = new Box3D(2, 3, 4);
        System.out.println(d.area());
        System.out.println(d.volume());
    }
}

```

7. Multiple Catch Blocks

```

class M {
    public static void main(String[] a) {
        try {
            int x = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Math error");
        } catch (Exception e) {
            System.out.println("Other error");
        }
    }
}

```

8. Try-Catch-Finally

```

class TCF {
    public static void main(String[] a) {
        try {
            int x[] = new int[2];
            x[5] = 10;
        } catch (Exception e) {
            System.out.println("Error");
        } finally {
            System.out.println("End");
        }
    }
}

```

9. User-Defined Exception

```
class AgeEx extends Exception {
    AgeEx(String s) {
        super(s);
    }
}

class T {
    static void check(int a) throws AgeEx {
        if (a < 18)
            throw new AgeEx("Not eligible");
    }

    public static void main(String[] a) {
        try {
            check(15);
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

10. Write File (OutputStream)

```
import java.io.*;

class W {
    public static void main(String[] a) throws Exception {
        FileOutputStream f = new FileOutputStream("out.txt");
        f.write("Hello".getBytes());
        f.close();
    }
}
```

11. Write User Input to File

```
import java.util.*;
import java.io.*;

class W2 {
    public static void main(String[] a) throws Exception {
        Scanner s = new Scanner(System.in);
        FileWriter w = new FileWriter("user.txt");
        w.write(s.nextLine());
        w.close();
    }
}
```

12. Payslip Calculation

```

class Emp {
    double bp;

    Emp(double b) {
        bp = b;
    }

    void slip() {
        double da = 0.97 * bp;
        double hra = 0.1 * bp;
        double pf = 0.12 * bp;
        double sc = 0.001 * bp;

        double gross = bp + da + hra;
        double net = gross - (pf + sc);

        System.out.println("Gross=" + gross + " Net=" + net);
    }

    public static void main(String[] a) {
        new Emp(50000).slip();
    }
}

```

13. Prime Number Check (Interface)

```

interface P {
    boolean c(int n);
}

class T implements P {
    public boolean c(int n) {
        if (n < 2)
            return false;
        for (int i = 2; i * i <= n; i++)
            if (n % i == 0)
                return false;
        return true;
    }

    public static void main(String[] a) {
        System.out.println(new T().c(17));
    }
}

```

14. Inheritance Hierarchy

```
class A {
    void a() {
        System.out.println("A");
    }
}

class B extends A {
}

class X {
    void x() {
        System.out.println("X");
    }
}

class Y extends X {
}

class Z extends Y {
}

interface I {
    void i();
}

class Base {
    void b() {
        System.out.println("B");
    }
}

class H extends Base implements I {
    public void i() {
        System.out.println("I");
    }

    public static void main(String[] a) {
        new B().a();
        new Z().x();
        new H().b();
        new H().i();
    }
}
```

16. JavaFX MenuBar (Minimal)

```

import javafx.application.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;

public class M extends Application {
    public void start(Stage s) {
        MenuBar mb = new MenuBar();
        Menu m = new Menu("File");
        m.getItems().add(new MenuItem("Open"));
        mb.getMenus().add(m);

        s.setScene(new Scene(mb, 300, 200));
        s.show();
    }

    public static void main(String[] a) {
        launch(a);
    }
}

```

17. Submenu with JavaFX

```

import javafx.application.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;

public class SM extends Application {
    public void start(Stage s) {
        MenuBar mb = new MenuBar();
        Menu m = new Menu("Tools");
        Menu sub = new Menu("Convert");
        sub.getItems().add(new MenuItem("Decimal to Binary"));
        m.getItems().add(sub);
        mb.getMenus().add(m);

        s.setScene(new Scene(mb, 300, 200));
        s.show();
    }

    public static void main(String[] a) {
        launch(a);
    }
}

```

18.i GridPane Layout

```

import javafx.application.*;
import javafx.scene.*;
import javafx.scene.layout.*;
import javafx.scene.control.*;
import javafx.stage.*;

public class G extends Application {
    public void start(Stage s) {
        GridPane g = new GridPane();
        g.add(new Label("Name"), 0, 0);
        g.add(new TextField(), 1, 0);
        g.add(new Label("Age"), 0, 1);
        g.add(new TextField(), 1, 1);

        s.setScene(new Scene(g, 300, 200));
        s.show();
    }

    public static void main(String[] a) {
        launch(a);
    }
}

```

18.ii BorderPane Layout

```

import javafx.application.*;
import javafx.scene.*;
import javafx.scene.layout.*;
import javafx.scene.control.*;
import javafx.stage.*;

public class B extends Application {
    public void start(Stage s) {
        BorderPane b = new BorderPane();
        b.setTop(new Button("Top"));
        b.setCenter(new Button("Center"));
        b.setBottom(new Button("Bottom"));
        b.setLeft(new Button("Left"));
        b.setRight(new Button("Right"));

        s.setScene(new Scene(b, 400, 300));
        s.show();
    }

    public static void main(String[] a) {
        launch(a);
    }
}

```

19. Total + Average

```
import java.util.*;

class M {
    public static void main(String[] a) {
        Scanner s = new Scanner(System.in);
        double t = 0;

        for (int i = 0; i < 5; i++)
            t += s.nextDouble();

        System.out.println("Total=" + t + " Avg=" + (t / 5));
    }
}
```

19. Palindrome Check

```
class P {
    static boolean p(int n) {
        int r = 0, o = n;
        while (n > 0) {
            r = r * 10 + n % 10;
            n /= 10;
        }
        return o == r;
    }

    public static void main(String[] a) {
        System.out.println(p(121));
    }
}
```

20. Abstract Shape

```
abstract class S {
    abstract void area();
}

class R extends S {
    int l, b;

    R(int l, int b) {
        this.l = l;
        this.b = b;
    }

    void area() {
        System.out.println(l * b);
    }

    public static void main(String[] a) {
        new R(5, 4).area();
    }
}
```

21. Fibonacci Series

```
class F {
    public static void main(String[] a) {
        int n = 5, x = 0, y = 1;

        for (int i = 0; i < n; i++) {
            System.out.print(x + " ");
            int t = x + y;
            x = y;
            y = t;
        }
    }
}
```

21. Factorial

```
class Fa {
    static int f(int n) {
        int r = 1;
        for (int i = 1; i <= n; i++)
            r *= i;
        return r;
    }

    public static void main(String[] a) {
        System.out.println(f(5));
    }
}
```

22. Wait & Notify (Threading)

```
class Acc {
    int amt = 10000;

    synchronized void wd(int w) {
        while (amt < w) {
            try {
                wait();
            } catch (Exception e) {
            }
        }
        amt -= w;
        System.out.println("Withdrawal: " + w);
    }

    synchronized void dep(int d) {
        amt += d;
        System.out.println("Deposit: " + d);
        notify();
    }
}

class T {
    public static void main(String[] a) {
        Acc c = new Acc();

        new Thread(() -> c.wd(15000)).start();
        new Thread(() -> {
            try {
                Thread.sleep(2000);
            } catch (Exception e) {
            }
            c.dep(10000);
        }).start();
    }
}
```

Key Indentation Rules Used:

- 4 spaces per indentation level
- Methods and fields indented within class
- Loop bodies indented within loops
- Try-catch-finally blocks properly aligned
- Import statements at top without indentation